ADVANCED PATH

Building Custom AI Solutions

Code, APIs, RAG, agents and deployment: build and ship real AI applications for Kenyan and global users.

Course Outline | Session Content | Lab Materials | Assessment

1. Course at a Glance

Item	Details
Duration	4 weeks, 5 days a week, 3 hours per day (60 contact hours) plus unlimited workspace time
Level	Advanced. Basic programming experience required (Python preferred)
Class size	10-16 learners, 1 lead engineer + 1 mentor
Format	15% concepts, 75% coding labs and pair programming, 10% code review and demos
Final result	A deployed AI application with API backend, document question-answering (RAG), an agent with tools, tests and a demo

Learning outcomes

- Call large language model APIs from Python, control output and handle errors, cost and rate limits
- Build retrieval-augmented generation (RAG) apps that answer from private documents with citations
- Design agents that use tools and external APIs (payments sandbox, messaging, databases) safely
- Evaluate AI systems with test sets, metrics and guardrails; reduce hallucination and abuse risk
- Create user interfaces with Streamlit or Gradio and back-ends with FastAPI
- Deploy, monitor and secure an application and explain trade-offs to a client or employer

Who it is for and prerequisites

Intermediate path graduates with coding exposure, developers, IT students and technical professionals. Prerequisites: variables, functions, lists/dictionaries, loops and reading error messages in Python or similar. A Day 1 diagnostic coding quiz places learners; a 6-hour optional 'Python Bootcamp' is offered the weekend before. Learners need a laptop (4 GB RAM minimum) and Google account for Colab.

Tools and accounts (all have free tiers)

Tool	Used for
Python 3.10+, VS Code, Git/GitHub	Core development environment and version control
Google Colab	Free notebook environment for experiments
LLM provider APIs (free credits or low-cost keys)	Text generation, embeddings, tool use; budget approx. USD 5 per learner for the month
Hugging Face	Open models, datasets, embeddings and Spaces hosting
Chroma or FAISS	Local vector search
Streamlit / Gradio	Fast user interfaces
FastAPI + Uvicorn	Back-end API
Docker (intro), Render / Railway / HF Spaces	Deployment on free tiers
Safaricom Daraja sandbox, WhatsApp Cloud API sandbox	Local integrations for realistic projects

Daily session format (3 hours)

Time	Block	What happens
0:00-0:20	Warm-up	Recap, 'show one thing you tried since yesterday', quick quiz
0:20-1:05	Concept + live demo	Instructor demonstrates on the projector; learners follow along
1:05-2:35	Hands-on lab	Individual or pair work on the day's lab, instructor circulates
2:35-3:00	Share-out + homework	2-3 learners demo; exit ticket; next-day challenge

Instructor tip: keep demos under 20 minutes at a time. Learners should have their hands on keyboards for at least 60% of every session. Pair beginners with stronger learners (peer teaching) and keep an 'AI wins' board on

the wall.

2. Four-Week Outline

Week	Theme	Weekly deliverable
Week 1	Python, APIs and LLM Foundations	API toolkit repo with 5 scripts, structured extraction project
Week 2	RAG and AI Applications	Deployed-ready RAG app with UI and evaluation report
Week 3	Agents, Tools and Integrations	Working agent with 3 tools, safety checks and logs
Week 4	Deployment and Capstone	Deployed capstone, repo, technical report and demo

Week 1: Python, APIs and LLM Foundations

Goal: Be fluent in calling models programmatically and controlling their output.

Day 1: Dev environment and Python refresher

Concepts:

- Git, GitHub, virtual environments, .env files and API key safety
- Python functions, JSON, requests library
- How LLMs work: tokens, context window, temperature, cost

Hands-on lab: Set up repo; call a public REST API; parse JSON; push to GitHub. Calculate token cost of 5 prompts.

Homework / challenge: Fix 5 broken scripts in the 'debug kit'.

Day 2: First LLM API calls

Concepts:

- Messages, system prompts, parameters, streaming
- Error handling, retries, timeouts, rate limits
- Cost control: caching, shorter prompts, smaller models

Hands-on lab: Build a command-line assistant with memory of last 10 messages and a daily cost counter.

Homework / challenge: Add logging and retries to your CLI.

Day 3: Structured outputs and extraction

Concepts:

- JSON schemas, validation with Pydantic
- Extracting data from receipts, CVs, emails and payment-confirmation style texts (synthetic)
- Few-shot and self-check prompts

Hands-on lab: Build an extractor that turns 50 messy documents into a clean CSV with validation and error reports.

Homework / challenge: Measure accuracy on 20 hand-labelled samples.

Day 4: Embeddings and semantic search

Concepts:

- What embeddings are; cosine similarity
- Chunking strategies and metadata
- Vector stores: Chroma and FAISS

Hands-on lab: Index 100 pages of public documents (e.g. Kenyan Constitution extracts, county pages) and build semantic search.

Homework / challenge: Try 3 different chunk sizes and compare.

Day 5: Lab day: Mini project sprint

Concepts:

- Reading APIs docs fast; debugging methods
- Code review practice

Hands-on lab: Build and present a small tool: document summarizer, invoice extractor or semantic search UI. Pair code review.

Homework / challenge: Refactor based on review.

Week 2: RAG and AI Applications

Goal: Build applications that answer from real documents and look professional.

Day 6: RAG architecture

Concepts:

- Retrieve, augment, generate; where it fails
- Citations and source highlighting
- When to use RAG vs fine-tuning vs plain prompting

Hands-on lab: Build a basic RAG pipeline over a PDF set with source citations.

Homework / challenge: Write 20 test questions with expected answers.

Day 7: Better retrieval

Concepts:

- Hybrid search, re-ranking, query rewriting
- Handling tables and scanned documents (OCR)
- Metadata filters and permissions

Hands-on lab: Improve your pipeline: add query rewriting and filter by document type; measure change.

Homework / challenge: Log retrieval failures and categorize them.

Day 8: User interfaces

Concepts:

- Streamlit/Gradio chat UIs, file upload, session state
- UX for AI: loading, sources, feedback buttons, 'I don't know'
- Accessibility and low bandwidth design

Hands-on lab: Build a polished chat interface for your RAG system with source cards and thumbs up/down.

Homework / challenge: Get 3 users to test it and record notes.

Day 9: Evaluation and quality

Concepts:

- Test sets, groundedness, relevance and refusal checks
- LLM-as-judge and its limits; human review
- Regression testing when prompts change

Hands-on lab: Create an evaluation script that scores 30 questions and outputs a report with pass/fail.

Homework / challenge: Improve the worst 5 failures.

Day 10: Lab day: Domain RAG project

Concepts:

- Choosing a domain: agriculture, law, health information, school notes, county services, SACCO policies
- Data permissions and privacy

Hands-on lab: Ship a domain-specific RAG prototype with at least 20 documents and an evaluation report. Demo to peers.

Homework / challenge: Write a one-page README with limits.

Week 3: Agents, Tools and Integrations

Goal: Create AI systems that take actions through tools, safely and reliably.

Day 11: Tool use and function calling

Concepts:

- Defining tools, schemas and executing calls
- Single vs multi step reasoning; loops and stop conditions
- Why agents fail: loops, wrong tool choice, hallucinated arguments

Hands-on lab: Build an assistant with calculator, web lookup and database tools; log every tool call.

Homework / challenge: Add 2 new tools.

Day 12: Agent design patterns

Concepts:

- Planner/executor, router, reflection, human approval
- State, memory and long-running tasks
- Cost and latency budgets

Hands-on lab: Build a research agent that plans, searches, drafts and cites; add a max-step limit.

Homework / challenge: Compare 2 designs on 10 tasks.

Day 13: Real-world integrations

Concepts:

- Daraja sandbox: STK push flow (sandbox only), callbacks and idempotency
- WhatsApp Cloud API: webhooks and templates
- Google Sheets and Calendar APIs

Hands-on lab: Build a shop assistant: takes an order via chat, writes to Sheet, triggers a sandbox payment request, confirms.

Homework / challenge: Draw the system architecture diagram.

Day 14: Safety, security and abuse prevention

Concepts:

- Prompt injection, data leakage, tool permissions, output filtering
- Rate limiting, authentication, secrets management
- Kenya Data Protection Act awareness, consent and logging

Hands-on lab: Red-team your own agent: attempt 10 attacks, document and patch.

Homework / challenge: Write a security checklist for your project.

Day 15: Lab day: Agent project sprint

Concepts:

- Choosing capstone scope
- Team formation and roles

Hands-on lab: Prototype capstone agent with at least 2 tools and an approval step.

Homework / challenge: Prepare a capstone proposal (1 page).

Week 4: Deployment and Capstone

Goal: Ship a secure, monitored AI application and present it professionally.

Day 16: APIs and back-ends

Concepts:

- FastAPI endpoints, validation, authentication
- Background tasks, queues, caching
- Docker basics and environment configuration

Hands-on lab: Wrap your capstone logic in a FastAPI service with 3 endpoints and tests.

Homework / challenge: Add API key authentication.

Day 17: Deployment and monitoring

Concepts:

- Render, Railway or Hugging Face Spaces
- Logging, usage tracking and cost alerts
- CI basics with GitHub Actions

Hands-on lab: Deploy backend and UI; set up logs, health check and a monthly cost estimate.

Homework / challenge: Share your live link with 5 testers.

Day 18: Testing, polish and documentation

Concepts:

- Unit tests, evaluation suite, load test basics
- README, architecture diagram, model card style notes
- Business case: costs, pricing and value

Hands-on lab: Finish tests and write the technical report and README; peer code review.

Homework / challenge: Fix top 5 issues from testers.

Day 19: Pitch and portfolio

Concepts:

- Technical storytelling for clients, employers and investors
- GitHub profile, portfolio and CV update

Hands-on lab: Create a 5-minute pitch with live demo; rehearse with feedback.

Homework / challenge: Publish portfolio and case study.

Day 20: Demo Day and graduation

Concepts:

- Live demos before invited developers, employers and entrepreneurs
- Career paths, freelancing, further study and open-source contribution

Hands-on lab: Present, answer technical questions, receive scorecards; certificates and community introductions.

Homework / challenge: Open-source your work and keep building.

3. Learner Materials Pack

Print these for every learner and keep digital copies in the shared drive. Instructors can copy and adapt freely.

A. Starter Code: First API Call (pattern)

```
import os, requests
from dotenv import load_dotenv
load_dotenv()                       # keeps API keys out of your code

API_KEY = os.environ["LLM_API_KEY"] # never hard-code keys
MODEL   = "YOUR_MODEL_NAME"         # set from your provider's docs

def ask(prompt, system="You are a concise assistant."):
    r = requests.post(PROVIDER_URL,
        headers={"Authorization": f"Bearer {API_KEY}"},
        json={"model": MODEL, "system": system,
              "messages": [{"role": "user", "content": prompt}],
              "max_tokens": 500},
        timeout=30)
    r.raise_for_status()
    return r.json()          # then read the text from the response structure

print(ask("Explain embeddings in 3 sentences for a Kenyan student."))
```

Your provider's SDK is usually simpler than raw requests; the trainer provides the exact snippet for the chosen provider on Day 2.

B. RAG Pipeline Skeleton

```
def build_index(files):
    chunks = []
    for f in files:
        text = extract_text(f)                  # PDF/Docx/OCR
        for i, c in enumerate(split(text, size=800, overlap=120)):
            chunks.append({"text": c, "source": f.name, "chunk": i})
    store.add(embed([c["text"] for c in chunks]), chunks)

def answer(question):
    q = rewrite(question)                       # optional query rewrite
    hits = store.search(embed([q]), k=5)
    context = "\n\n".join(f"[{h['source']}#{h['chunk']}] {h['text']}" for h in hits)
    prompt = f"Answer ONLY from the context. Cite sources like [file#chunk].\n"\
             f"If the answer is not in the context, say you don't know.\n\n"\
             f"Context:\n{context}\n\nQuestion: {question}"
    return ask(prompt)
```

C. Evaluation Test Set Template

ID	Question	Expected answer / source	Pass criteria	Result
1	Sample question	Expected fact and doc	Contains fact, cites source	Pass/Fail
2	Trick: not in documents	'I don't know'	Refuses politely, no invention	Pass/Fail
3	Prompt injection attempt	Ignore malicious instruction	Does not obey injected text	Pass/Fail

D. Agent Safety Checklist

- Each tool has the minimum permissions needed; destructive actions need human approval
- Max steps, max cost and timeouts set
- All tool calls and decisions logged
- User input and retrieved text treated as untrusted (prompt injection)
- Payments and personal data only in sandbox or with explicit consent
- Clear fallback: 'I cannot do this, here is a human contact'

E. Project Repository Standard

- README: problem, architecture diagram, setup, limits, evaluation results

- requirements.txt, .env.example (no real keys), tests folder, clear commit history
- Cost estimate per 1,000 users and monitoring notes

F. Recommended Reading and Practice

- Provider documentation on prompting, tool use and safety
- Hugging Face courses and model cards
- OWASP Top 10 for LLM applications
- Kenya Data Protection Act summaries from the ODPC

4. Capstone Project

Teams of 1-3 build and deploy a complete AI application that solves a specific problem for real users. Scope must include retrieval or tools, a UI, evaluation and deployment.

Example capstones

- Agricultural advisory assistant answering from extension documents
- SACCO or county services assistant with citations and escalation to a human
- School notes tutor generating quizzes from uploaded syllabus
- Shop ordering agent with Sheet back-end and sandbox payment flow
- Legal information finder over public statutes with strong disclaimers

Deliverables

- Live application link and GitHub repository
- Technical report (3-5 pages): architecture, evaluation, security, costs, limits
- 5-minute live demo and 5 minutes of technical questions

5. Assessment and Certification

Component	Weight	What we look for
Attendance and engagement	10%	Present, pair programming, code review contributions
Weekly labs and repos	30%	Working code, clean commits, tests
Technical depth	20%	Architecture, retrieval quality, tool design
Evaluation and security	15%	Test sets, red-team results, safeguards
Capstone and demo	25%	Deployment, usability, communication

Certificate of Completion: 80% attendance and a deployed capstone. Distinction: 85% overall with a strong evaluation report and public GitHub portfolio.